Sorted() function - Used to sort collections.

```
l = [5, 1, 2, 3]
print(sorted(l))   # [1, 2, 3, 5]
print(l)   # [5, 1, 2, 3]
l.sort()
print(l)   # [1, 2, 3, 5]
```

* Doesn't alter the original list.
* Returns the output.

Lambda function : A small (one-line) function without a name.

```
def square(x):
    return x*x

square = lambda x : x*x
print(square(5))  # 25
```

Syntax: lambda <input> : <output>

```
def add(x, y):
    return x+y.

add = lambda x,y : x+y

lambda s : s[0]   # Returns first character of the word.
lambda n : n%2==0   # Check if a number is even
lambda s : s.upper()   # Converts string to uppercase.
```

```
get_last = lambda s: s[-1]
print(get_last("Scaler"))   # r
```

<u>sorted() using lambda</u>

```
words = ["apple", "kiwi", "banana"]
print(sorted(words))   # Normal
print(sorted(words, key = lambda s: len(s)))
```

Defines the sorting parameter

```
# ["kiwi", "apple", "banana"]
```

4 5 6

```
keys = []
for word in words:
    keys.append(len(word))
#Sort based on keys.
```

```
num = [-5, -1, 3, 2, -4]
print(sorted(num, key = lambda n: abs(n)))
# [-1, 2, 3, -4, -5]
```

```
words = ["apple", "banana", "cherry"]
print(sorted(words, key = lambda w: w[-1]))
# ["banana", "apple", "cherry"]
```

[a, e, y] ✗ - Sort parameter.

```
pairs = [(1, 5), (2, 3), (4, 1), (6, 5)]
print(sorted(pairs, key = lambda t: t[1]))
# Sort keys = [5, 3, 1]
# Output = [(4, 1), (2, 3), (1, 5), (6, 5)]
```

```
students = [{"name": "Alice", "score": 88},
            {"name": "Bob", "score": 95},
            {"name": "Charlie", "score": 70}]
print(sorted(students, key = lambda d: d["score"]))
```

Timestamps : Date + time entry of the event.

2026-03-06 16:40 : User entered.

2026-03-06 16:40 : User entered.

* Use UTC time for timestamps.

Log entry : Record of the events.

2026-03-06 16:30:00 - INFO Instructor started class.

2026-03-06 16:30:10 - INFO Students join

2026-03-06 16:35:00 - Instructor starts class.

- Shares screen.

- . - .

ERROR - Problem started.

Log entry

- Timestamp.
- Type of message (INFO, WARNING, ERROR, FATAL)
- Comment.

Question: Given a company's logs from multiple servers at multiple events, combine them and sort them by timestamps.

ex:

```
logs = {
    'server1_logs' : [
        {'timestamp' : '2024-07-18  10:04:10',
         'level': 'ERROR',
         'message' : 'Disk full'},
        {'Timestamp' : '2024-07-18  10:00:01',
         'level' : 'INFO',
         'message' : 'Server Started'},]
    'server2_logs' : [
        {'timestamp' : '2024-07-18  10:01:17',
         'level': 'ERROR',
         'message' : 'Database connection failed'},
        {'Timestamp' : '2024-07-18  10:02:45',
         'level' : 'WARNING',
         'message' : 'High Memory'},]
}
```

Approach

- Go over each key (server name) and get all logs into a common list.
- Write lambda function to get timestamp from each log.
- Sort based on the timestamp.

```
def logSort(logs):
    common_logs = []
    for server_name in logs:
        server_logs = logs[server_name]
        # list of each servers logs.
        common_logs += server_logs.
        # Concat all logs together.

    return sorted(common_logs,
                  key = lambda d: d["timestamp"])
```

Each log entry = { "timestamp" : - - .
"level" : - -
"message" : - - - }